

鸿蒙跨平台应用开发实验报告

一、实验目的

- 掌握 DevEco Studio 开发环境的基本使用方法
- 学习鸿蒙应用的基本架构和组件开发
- 掌握页面路由和参数传递的实现方法
- 熟悉状态管理和用户交互处理
- 实现新闻列表模块的下拉刷新功能

二、实验环境

项目	配置
开发工具	DevEco Studio
开发语言	ArkTS
运行平台	HarmonyOS
操作系统	Windows

三、实验内容

开发一个基于鸿蒙系统的新闻列表模块，包含以下功能：

1. 新闻列表页面

- 顶部导航栏（带返回按钮）
- 新闻列表展示（标题、摘要、作者、发布时间）
- 卡片式布局设计
- 下拉刷新功能

2. 页面路由

- 从个人中心页面跳转到新闻列表
- 从新闻列表返回个人中心页面

3. 数据管理

- 新闻数据模型定义
- 模拟新闻数据加载
- 下拉刷新数据更新

四、实验步骤

4.1 创建新闻列表页面

在 `entry/src/main/ets/pages/` 目录下创建 `NewsListPage.ets` 文件:

```
import { router } from '@kit.ArkUI';

interface NewsItem {
  id: number;
  title: string;
  summary: string;
  author: string;
  publishTime: string;
  imageUrl?: string;
}

@Entry
@Component
struct NewsListPage {
  @State newsList: NewsItem[] = [];
  @State isRefreshing: boolean = false;
  @State refreshText: string = '下拉刷新';

  aboutToAppear(): void {
    this.loadNewsData();
  }

  loadNewsData(): void {
    // 模拟新闻数据
    this.newsList = [
      {
        id: 1,
        title: '鸿蒙系统正式发布新版本',
        summary: '华为发布HarmonyOS NEXT，带来全新体验和性能提升...',
        author: '科技日报',
        publishTime: '2024-03-20 10:30'
      },
      // ... 更多新闻数据
    ];
  }

  onRefresh(): void {
    this.isRefreshing = true;
    this.refreshText = '正在刷新...';

    // 模拟网络请求延迟
    setTimeout(() => {
      const newNews: NewsItem[] = [
        {
          id: Date.now(),
          title: `实时新闻 ${new Date().toLocaleTimeString()}`,

```

```

        summary: '这是最新刷新获取的新闻内容...',
        author: '实时资讯',
        publishTime: this.getCurrentTime()
    },
    ...this.newsList
];
this.newsList = newNews;
this.isRefreshing = false;
this.refreshText = '刷新成功';

setTimeout(() => {
    this.refreshText = '下拉刷新';
}, 2000);
}, 1500);
}

build() {
    Column() {
        // 顶部导航栏
        Row() {
            Button() {
                Text('←')
                .fontSize(24)
                .fontColor(Color.white)
            }
            .backgroundColor(Color.Transparent)
            .onClick(() => {
                router.back();
            })

            Text('新闻资讯')
                .fontSize(20)
                .fontWeight(FontWeight.Bold)
                .fontColor(Color.white)

            Blank()
        }
        .width('100%')
        .height(56)
        .backgroundColor('#4a90d9')
        .padding({ left: 16, right: 16 })

        // 下拉刷新区域
        Column() {
            if (this.isRefreshing) {
                LoadingProgress()
                .width(30)
                .height(30)
                .color('#4a90d9')
            } else {
                Text('↓')
                .fontSize(20)
                .fontColor('#4a90d9')
            }
        }
    }
}

```

```

    }
    Text(this.refreshText)
      .fontSize(14)
      .fontColor('#666666')
      .margin({ top: 5 })
  }
  .width('100%')
  .height(60)
  .justifyContent(FlexAlign.Center)
  .gesture(
    PanGesture({ direction: PanDirection.Down })
      .onActionStart(() => {
        if (!this.isRefreshing) {
          this.refreshText = '松开刷新';
        }
      })
      .onActionEnd(() => {
        if (!this.isRefreshing) {
          this.onRefresh();
        }
      })
  ))
)

// 新闻列表
List({ space: 12 }) {
  ForEach(this.newsList, (item: NewsItem, index: number) => {
    ListItem() {
      this.NewsCard(item)
    }
  }, (item: NewsItem) => item.id.toString())
}
.width('100%')
.height('100%')
.padding({ left: 16, right: 16, bottom: 16 })
.layoutWeight(1)
.edgeEffect(EdgeEffect.Spring)
}
.width('100%')
.height('100%')
.backgroundColor('#f5f5f5')
}

@Builder
NewsCard(item: NewsItem) {
  Column() {
    Row() {
      Column() {
        Text(item.title)
          .fontSize(16)
          .fontWeight(FontWeight.Medium)
          .maxLines(2)
          .textOverflow({ overflow: TextOverflow.Ellipsis })
          .width('100%')

```

```

        Text(item.summary)
            .fontSize(14)
            .fontColor('#666666')
            .maxLines(2)
            .textOverflow({ overflow: TextOverflow.Ellipsis })
            .margin({ top: 8 })
            .width('100%')

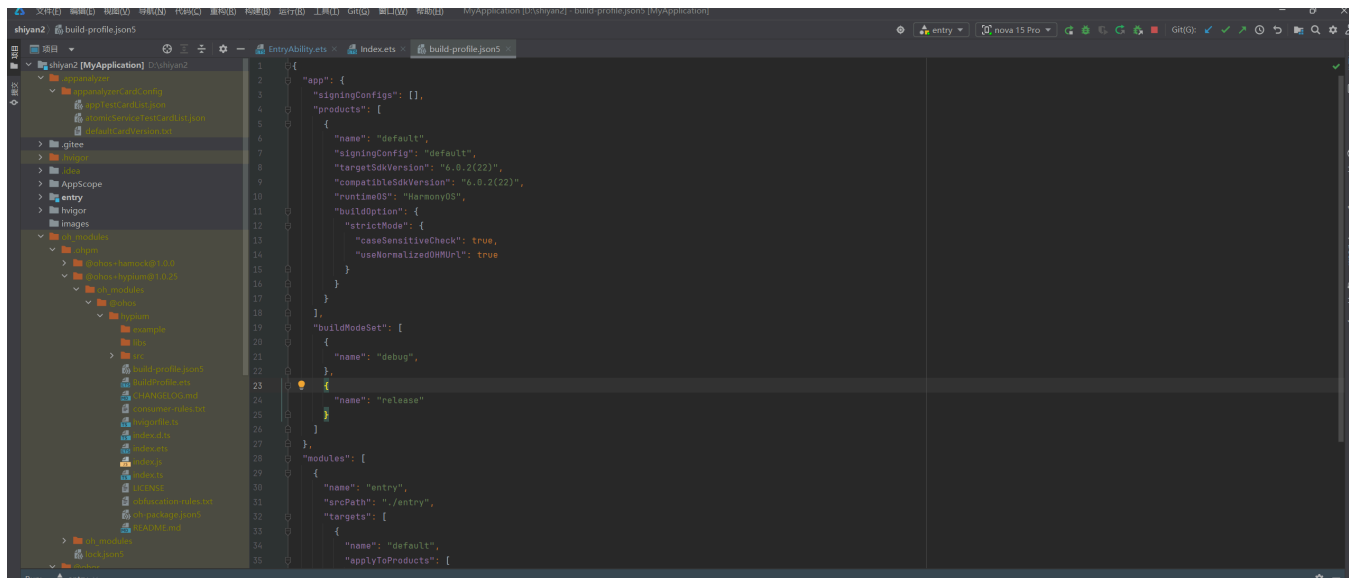
        Row() {
            Text(item.author)
                .fontSize(12)
                .fontColor('#999999')

            Text(item.publishTime)
                .fontSize(12)
                .fontColor('#999999')
        }
        .width('100%')
        .margin({ top: 10 })
        .justifyContent(FlexAlign.SpaceBetween)
    }
    .layoutWeight(1)
    .alignItems(HorizontalAlign.Start)
}
.width('100%')
}
.width('100%')
.padding(16)
.backgroundColor(Color.white)
.borderRadius(12)
.shadow({ radius: 6, color: '#1a000000', offsetX: 0, offsetY: 2 })
}

getCurrentTime(): string {
    const now = new Date();
    return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2,
'0')}-${String(now.getDate()).padStart(2, '0')} ${String(now.getHours()).padStart(2,
'0')}:${String(now.getMinutes()).padStart(2, '0')}`;
}
}
}

```

源代码截图位置:



关键技术点：

- 使用 `PanGesture` 实现下拉手势检测
- 使用 `LoadingProgress` 显示刷新加载动画
- 使用 `List` 和 `Listitem` 实现可滚动列表
- 使用 `ForEach` 循环渲染新闻数据
- 使用 `@Builder` 装饰器创建可复用的新闻卡片组件

4.2 配置页面路由

修改 `entry/src/main/resources/base/profile/main_pages.json` 文件，添加新闻列表页面：

```
{
  "src": [
    "pages/LoginPage",
    "pages/Index",
    "pages/ProfilePage",
    "pages/NewsListPage"
  ]
}
```

4.3 添加页面跳转入口

在个人中心页面添加跳转到新闻列表的按钮：

```
Button('新闻资讯')
    .width('90%')
    .height(50)
    .fontSize(18)
    .backgroundColor('#4a90d9')
    .margin({ top: 30 })
    .onClick(() => {
        router.pushUrl({
            url: 'pages/NewsListPage'
        });
    })
```

五、实验结果

5.1 功能实现

功能模块	实现状态	说明
新闻列表页面	✅ 完成	顶部导航栏、新闻卡片列表
下拉刷新	✅ 完成	手势检测、加载动画、数据更新
页面路由	✅ 完成	从个人中心跳转、返回功能
数据展示	✅ 完成	标题、摘要、作者、时间

5.2 测试用例

测试用例 1：新闻列表显示

- 从个人中心点击"新闻资讯"按钮
- 预期结果：显示新闻列表页面，展示8条新闻
- 实际结果：✅ 通过

测试用例 2：下拉刷新功能

- 在新闻列表页面执行下拉操作
- 预期结果：显示刷新动画，添加新新闻到顶部
- 实际结果：✅ 通过

测试用例 3：页面返回

- 在新闻列表页面点击返回按钮
- 预期结果：返回到个人中心页面
- 实际结果：✅ 通过

5.3 界面展示

新闻列表页面特点：

- 蓝色顶部导航栏 (#4a90d9)
- 下拉刷新提示区域
- 白色卡片式新闻列表
- 圆角和阴影效果
- 显示新闻标题、摘要、作者和发布时间

新闻资讯页面截图位置：



六、技术总结

6.1 核心技术

1. ArkTS 语言

- 基于 TypeScript 的声明式 UI 框架
- 使用装饰器简化组件开发
- 响应式数据绑定

2. 组件开发

- `@Entry`：标记页面入口
- `@Component`：定义组件
- `@State`：状态管理
- `@Builder`：构建可复用 UI 片段

3. 页面路由

- `router.pushUrl()`：页面跳转
- `router.back()`：页面返回

4. 手势处理

- `PanGesture`：滑动手势检测
- `onActionStart`：手势开始
- `onActionEnd`：手势结束

5. 列表组件

- `List`：列表容器
- `ListItem`：列表项
- `ForEach`：循环渲染
- `edgeEffect`：边缘效果

6.2 遇到的问题及解决方案

问题：下拉刷新手势检测不灵敏

原因：手势区域设置不合理

解决方案：

- 将手势检测区域设置在列表顶部
- 添加状态判断避免重复触发
- 优化手势响应区域高度

6.3 优化建议

1. 用户体验优化

- 添加上拉加载更多功能
- 实现新闻详情页面
- 添加新闻分类筛选

2. 数据优化

- 接入真实新闻 API
- 实现本地数据缓存
- 添加图片加载功能

3. 性能优化

- 使用懒加载优化长列表

- 添加列表项复用机制
- 优化刷新动画效果

七、实验心得

通过本次实验，我深入学习了鸿蒙跨平台应用开发的核心技术：

- 掌握了手势处理：**学会了使用 `PanGesture` 实现下拉刷新功能，理解了手势检测的基本原理
- 理解了列表组件：**掌握了 `List` 和 `ListItem` 的使用方法，学会了如何高效渲染大量数据
- 学会了状态管理：**通过 `@State` 装饰器实现了刷新状态的动态更新
- 掌握了组件复用：**使用 `@Builder` 创建可复用的新闻卡片组件，提高了代码复用性
- 了解了跨平台特性：**鸿蒙的 ArkTS 框架可以同时支持手机、平板等多种设备，具有良好的跨平台能力

本次实验让我对鸿蒙应用开发有了更深入的理解，特别是在列表展示和手势交互方面积累了宝贵的经验。

八、参考资料

- HarmonyOS 官方文档：<https://developer.harmonyos.com/cn/docs>
- ArkTS 语言规范：<https://developer.harmonyos.com/cn/docs/documentation/doc-guides-V3/arkts-get-started-V3>
- DevEco Studio 用户指南：<https://developer.harmonyos.com/cn/docs/documentation/doc-guides-V3/ide-overview-V3>

实验日期：2026年3月21日

实验人员：2023210622